Continuous Integration

Cohort 3 Team 1: Pixels of Promise

Movitz Aaron, Jake Adams, Oliver Belam, Anna Burt, Emily Foran, Sky Haines-Bass, Job Young

University of York

ENG1: Software & Systems Engineering

January 13, 2025

a.)

PIPELINE 1 - Build, test, coverage, style check and artifact upload.

PIPELINE 1 automates building the project, running tests, checking code style, generating reports, and uploading artifacts. It compiles the code, runs tests to ensure everything functions correctly, and checks that the code follows Google style guidelines for better readability and consistency. The pipeline also creates reports on test coverage and code style, which are then uploaded for review. This process not only verifies that the build is successful but also highlights areas that may need more robust testing, ensuring maintainability, quality assurance, and quick feedback if anything breaks.

PIPELINE 1 is triggered when:

- Any push is made to any branch
- A pull request is made to the main branch
- A commit is pushed with a version tag. (A version tag must match the specified pattern `v[0-9]+.[0-9]+.[0-9]+`) .

PIPELINE 1's input is the current GitHub project.
PIPELINE 1's outputs are the JAR file of the game, with correct version number and test coverage and checkstyle reports.

PIPELINE 2 - GitHub dependency submission.

This is the dependency submission pipeline. It analyses the project dependencies and forwards the data to GitHub to create a dependency graph, detailing project dependencies, associated vulnerabilities and their severity. This is appropriate for the project as it ensures system security is monitored and consistently assessed, allowing for vulnerabilities to be analysed and preventative measures put in place before potential exploitation occurs.

PIPELINE 2 is triggered when PIPELINE 1 is successfully executed.

PIPELINE 2's input is the current project (who will build successfully thanks to PIPELINE 1).
PIPELINE 2's output is the dependency graph.

PIPELINE 3 - Deployment of the game to GitHub releases. (CD)

This is the deployment pipeline, responsible for downloading the .Jar from PIPELINE 1 and putting it into a GitHub release. This is necessary for the project as it automates the version control system, improving the project's maintainability since we can test a version with ease, no building of the project required, just download the newest .Jar version.

PIPELINE 3 is triggered when a push contains a version tag and PIPELINE 1 successfully executes.

PIPELINE 3's input is the successful .Jar artifact produced by PIPELINE 1 and an associated appropriate version tag, as well as the current GitHub project.
PIPELINE 3's output is a draft release, created on GitHub for manual review and publication.

b.) For our project we are using GitHub actions to complete all of our CI/CD pipelines and workflows, utilising the markup language YAML to configure our pipelines to be used by GitHub to execute them.

PIPELINE 1: Build Job

Key Steps:

1. **Checkout Repository**: Uses the `actions/checkout@v4` action to pull the latest code from the repository.
2. **Setup JDK 17**: Configures the Java Development Kit (JDK) version 17 from the Temurin distribution using `actions/setup-java@v4`. Ensures a consistent Java environment for the build.
3. **Setup Gradle**: Optimises Gradle usage by caching dependencies and configuration using `gradle/actions/setup-gradle`.
4. **Build and Test**: Runs `./gradle build` to compile the project, execute tests, and generate Jacoco test coverage and Checkstyle reports.
5. **Extract Project Version**:
    a. Invokes a Gradle task (`printVersion`) to extract current project version, from `gradle.properties`
    b. The version is printed to the console for a manual check, assigned to an environmental variable and stored in the job output `PROJECT_VERSION`, this is used by later jos to reference the .Jar artifact so that it can be downloaded.
6. **Upload artifacts**: Uploads all artifacts that were generated: JAR file, test report, Jacoco report and Checkstyle report.
    a. In the YAML: `continue-on-error: true` is included so that even if the upload fails (due to storage limitations) the build will succeed.

PIPELINE 2: Dependency Submission Job

Key steps:

1. **Checkout and Setup**: Similar to Build, checkout the repository and setup JDK 17.
2. **Generate and Submit Dependency Graph**: Uses the `gradle/actions/dependency-submission` action to: analyse all project dependencies, submit a dependency graph to GitHub and enables Dependabot alerts to auto-notify any detected vulnerabilities, ensuring security.

PIPELINE 3: Release Job

Key steps:

1. **Checkout and Setup**: Similar to PIPELINE 1 and PIPELINE 2
2. **Download Artifact**: Uses `actions/download-artifact` to fetch the JAR file produced in the Build job, the artifact name uses the `PROJECT_VERSION` environment variable to fetch the correct version.
3. **Release creation**: Confirms the JAR's presence, executes GitHub CLI (`gh`) commands to create a new draft release:
    a. Names the release tag using the format `v{PROJECT_VERSION}`
    b. Attaches the downloaded JAR
    c. Automatically generate release notes
    d. Upload as a draft for manual review.